GS-DroneGym: Photorealistic Simulation and Synthetic Dataset Generation for VLA Drone Navigation

Atul Saxena

Abstract

Vision-language-action (VLA) models are increasingly being explored for aerial robots, but drone navigation research requires infrastructure that connects language-conditioned visual observations to physically meaningful flight actions. Recent aerial VLA systems motivate high-fidelity visual data, staged training, waypoint-style action interfaces, and geometric safety correction, yet reusable open tools for generating and inspecting such data remain limited. We present GS-DroneGym, an open-source Python research stack for drone VLA navigation that combines 6-DOF quadrotor dynamics, Gymnasium-compatible tasks, waypoint control, 3D Gaussian Splatting rendering, live visualization, and synthetic waypoint-supervised dataset generation. The dataset factory exports RGB, depth, drone state, language instruction, expert waypoint, safety labels, and task metadata as Parquet shards with external media files. GS-DroneGym is designed as research infrastructure rather than a physical drone deployment stack. The package is released on PyPI and GitHub, and the current implementation includes CPU-testable mock rendering, optional `gsplat`-based rendering, five benchmark tasks, cross-benchmark data adapters, and command-line tools for dataset generation, validation, training, and evaluation.

1 Introduction

Vision-language-action models promise a convenient interface for robots: a model observes the world, receives an instruction, and emits an action. In manipulation, recent open VLA systems such as OpenVLA have made this interface more accessible by releasing model checkpoints and training code for generalist robot policies [3]. Aerial robots, however, impose a different set of constraints. Drone policies must operate under fast dynamics, limited onboard compute, strict safety requirements, and visual conditions that often differ substantially between synthetic simulation and real deployment spaces.

Recent aerial VLA work makes this gap explicit. VLA-AN proposes an onboard VLA framework for drone navigation and identifies several major limitations for aerial navigation models, including data domain gap, temporal reasoning, safety risks in generative action policies, and deployment constraints [6]. Its proposed ingredients include 3D Gaussian Splatting data, a progressive three-stage training framework, waypoint-oriented actioning, and geometric safety correction. RaceVLA separately studies VLA-based racing drone navigation and reports strong generalization in some axes while also showing reduced visual and physical generalization in dynamic racing settings [5]. Together, these works suggest that drone VLA research needs reusable infrastructure for photorealistic observation generation, task diversity, waypoint supervision, safety-aware labels, and standardized evaluation.

GS-DroneGym is built to provide such infrastructure. It is a Gymnasium-compatible drone research environment with quadrotor dynamics, camera modeling, rendering interfaces, task definitions, waypoint control, shared trajectory schemas, and a synthetic dataset factory. Its central

design choice is to treat waypoint outputs as the action interface for VLA-style policies. Instead of directly predicting raw motor commands, a policy can emit a target waypoint $[x, y, z, \psi]$, which is converted into low-level thrust and body-rate commands by a controller. This mirrors the abstraction used by several practical aerial navigation systems while keeping the environment usable for imitation learning, reinforcement learning, and offline dataset generation.

This paper describes the current GS-DroneGym system and its dataset-generation pipeline. The emphasis is on open infrastructure, not on claiming a new state-of-the-art policy. The current implementation is validated as a software artifact with unit tests, command-line tools, examples, PyPI packaging, and CPU-compatible mock rendering. The real 3D Gaussian Splatting path is supported through `gsplat`, but full real-scene dataset generation depends on external reconstruction pipelines and compatible GPU environments.

Contributions. This work contributes:

- A drone-first Gymnasium environment with 6-DOF quadrotor dynamics, waypoint and direct action modes, a PID waypoint controller, five navigation tasks, and a live viewer.
- A rendering abstraction with a deterministic CPU mock renderer for tests and an optional `gsplat`-based renderer for 3D Gaussian Splatting scenes.
- A VLA-AN-like synthetic dataset factory that generates RGB, depth, state, instruction, expert waypoint, safety labels, and metadata in Parquet-plus-media format.
- A shared trajectory and benchmark layer that supports GS-DroneGym rollouts and adapters for LIBERO and LeRobot-format data.

2 Related Work

Vision-language-action models for robotics. VLA models connect visual perception, language conditioning, and robot action generation. OpenVLA demonstrates that open VLA models can be trained on large-scale robot demonstration data and fine-tuned for new settings [3]. While this line of work has substantially advanced manipulation policies, aerial robotics differs in embodiment, dynamics, risk profile, and observation geometry. GS-DroneGym therefore focuses on the environment and data layer needed to study drone-specific VLA policies rather than introducing a new foundation model.

Aerial VLA navigation. VLA-AN and RaceVLA are the most direct motivations for this project. VLA-AN proposes an onboard aerial VLA framework and explicitly combines high-fidelity 3DGS data, staged training, an efficient action module, and geometric safety correction [6]. RaceVLA adapts VLA-style modeling to racing drone navigation and reports human-like behavior and generalization results across several axes [5]. GS-DroneGym does not reproduce either private dataset or trained model. Instead, it implements an open, reusable approximation of several public design ingredients: 3DGS-compatible rendering, staged task generation, waypoint supervision, and safety labels.

Photorealistic rendering with 3D Gaussian Splatting. 3D Gaussian Splatting represents radiance fields as Gaussian primitives and enables high-quality novel-view rendering [2]. This representation is attractive for sim-to-real research because scenes can be reconstructed from images

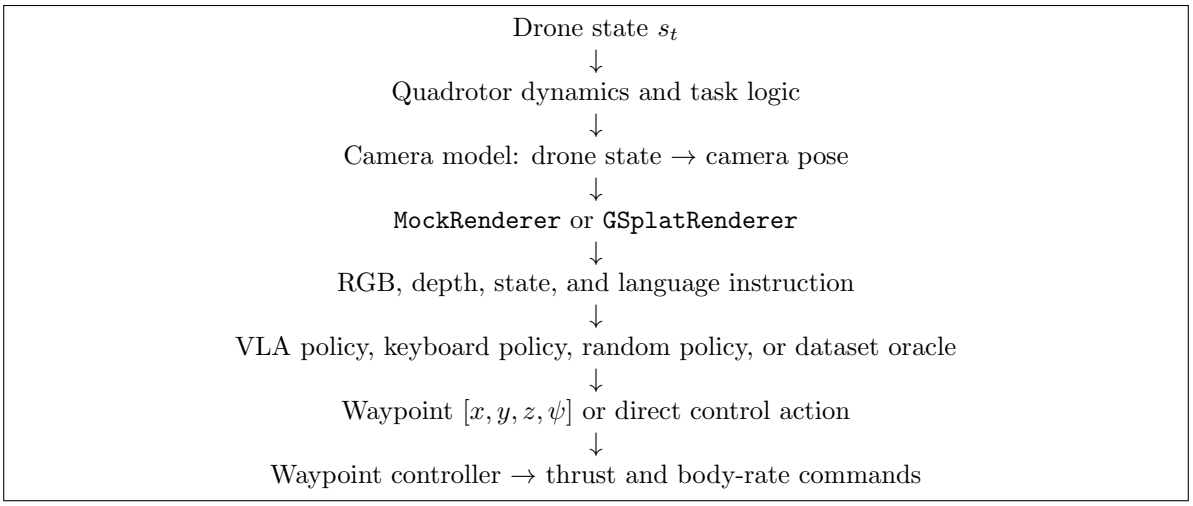

Figure 1: Closed-loop data flow in GS-DroneGym. The same environment can be used interactively, as a Gymnasium-style benchmark, or as a generator for waypoint-supervised synthetic data.

of real spaces and rendered from novel camera poses. GS-DroneGym exposes this idea through a renderer interface that can load Gaussian `.ply` scenes for drone camera observations while retaining a CPU mock renderer for testing and development.

Robot learning benchmarks and data formats. LIBERO provides a benchmark for lifelong robot learning and knowledge transfer [4]. LeRobot provides models, datasets, and tooling for real-world robotics, including a scalable dataset format based on video or images plus Parquet metadata [1]. GS-DroneGym adopts a compatible philosophy: keep embodiment-specific simulation separate from a shared trajectory representation so that live drone rollouts, generated aerial datasets, and external benchmark data can be inspected and trained over with common tools.

3 System Overview

GS-DroneGym is organized around a closed-loop drone simulation pipeline:

$$s_t \to \text{dynamics} \to \text{camera pose} \to \text{renderer} \to o_t \to \text{policy} \to \text{waypoint/controller} \to s_{t+1}.$$

The observation o_t contains an RGB image, optional depth image, 12-dimensional drone state, and a language instruction. The action can be interpreted in two modes. In direct mode, the action maps to collective thrust and body-rate commands. In waypoint mode, the action represents a relative displacement and yaw change, which is converted to a target waypoint and tracked by the waypoint controller.

3.1 Quadrotor Dynamics

The dynamics module represents the drone state as

$$[x, y, z, v_x, v_y, v_z, \phi, \theta, \psi, p, q, r],$$

where position and velocity are expressed in world coordinates, (ϕ, θ, ψ) are Euler angles, and (p, q, r) are angular rates. The implemented model uses configurable mass, gravity, first-order thrust

lag, physical action clipping, a lightweight linear drag term, ZYX Euler rotations, and Runge–Kutta integration. The simulation runs at a smaller dynamics timestep than the observation timestep, allowing several integration steps per environment transition.

Collision checks are deliberately simple in the current system. The environment supports axis-aligned scene bounds and task-provided obstacle primitives such as boxes and cylinders. This is sufficient for language-conditioned waypoint research where the primary labels are high-level motion targets, clearances, and collision indicators, but it is not a substitute for high-fidelity physical contact simulation. The package also includes optional visual, depth, and IMU perturbations for robustness experiments; these are augmentation utilities rather than a validated sensor model.

3.2 Waypoint Controller

The waypoint controller converts a VLA-style target $[x, y, z, \psi]$ into low-level thrust and rate commands. It uses a cascaded structure: position error defines desired velocity, velocity error defines desired acceleration, acceleration maps to thrust and attitude commands, and yaw is handled by a separate proportional loop. This controller is not intended to be an optimal flight controller. Its role is to provide a stable bridge between high-level waypoint labels and low-level simulated dynamics.

3.3 Rendering

GS-DroneGym exposes two renderer paths. The `MockRenderer` produces deterministic CPU RGB and depth observations for tests, examples, CI, and users without GPU support. The `GSplatRenderer` loads Gaussian Splatting `.ply` files and renders RGB, depth, and alpha through `gsplat`. This separation is important for reproducibility: the core environment can be tested without GPU dependencies, while researchers with Gaussian scenes can use photorealistic observations.

3.4 Tasks

The current task library includes five navigation tasks:

Task	Goal	Success condition
PointNav	Fly to a sampled 3D coordinate	within 0.5 m of goal
ObjectNav	Navigate to a named region	within 0.5 m of region goal
ObstacleSlalom	Traverse sequential gates	all gates passed
DynamicFollow	Track a moving target	within 1.0 m for 15 steps
NarrowCorridor	Fly through a tight corridor	reach corridor exit safely

Table 1: Built-in GS-DroneGym tasks. The tasks are intended to cover point navigation, language-conditioned region navigation, obstacle-aware motion, temporal tracking, and precision flight.

4 Synthetic Dataset Generation

The dataset factory generates staged waypoint-supervised data for aerial VLA research. It is inspired by the public ingredients described in VLA-AN—3DGS data, staged training, waypoint-style actioning, and geometric safety correction—but does not claim to reproduce VLA-AN's private dataset or training pipeline.

4.1 Curriculum Stages

The generator defines three stable stages:

- `stage1_scene_comprehension`: short grounding and pointing-style tasks.
- `stage2_flight_skills`: short-horizon flight skills, obstacle courses, corridor traversal, and recovery-like clips.
- `stage3_long_horizon_navigation`: longer navigation and dynamic-follow tasks.

The default curriculum is configurable, but the intended use is to produce data that gradually shifts from local scene understanding toward longer-horizon navigation behavior.

4.2 Labels and Output Format

Each generated step stores:

- language instruction,
- RGB image path,
- depth array path,
- 12-dimensional drone state,
- expert waypoint $[x, y, z, \psi]$,
- task and scene metadata,
- safety labels for imminent collision, minimum clearance, recovery requirement, occurred collision, and task success.

Large datasets are exported as Parquet step shards plus external media files. Debug episodes can also be exported using the common trajectory JSON schema. This layout is chosen to support both inspection and scalable training, following the general direction of modern robotics dataset tooling such as LeRobot [1].

4.3 Expert Waypoint Generation

The current expert is geometric rather than learned. It samples or computes task-relevant paths, emits dense global waypoints, and uses the waypoint controller for rollout. The expert waypoint is the primary supervision target; controller outputs are implementation details. Safety labels are computed from task geometry and obstacle clearance rather than inferred only from scalar reward.

5 Cross-Benchmark Data Layer

GS-DroneGym includes a shared trajectory schema with task, action, observation, step, episode, and benchmark-report objects. This schema is used for live drone rollouts, generated synthetic datasets, behavior cloning, and dataset inspection. Optional adapters support LIBERO and LeRobot-format data. The goal is not to force manipulation and aerial navigation into identical metrics; instead, the common layer standardizes episodes and artifacts while allowing benchmark-specific metrics to remain nested in reports.

Component	Current status	Evidence type
PyPI package	released as 0.3.0	installable package metadata
CPU mock environment	implemented	unit tests and viewer demos
3DGS renderer path	implemented, optional	import-gated tests and local integration path
Dataset factory	implemented	CLI, validation tests, Parquet/-media output
Benchmark adapters	implemented as optional layer	schema and adapter tests
Physical drone deployment	not implemented	explicitly out of scope

Table 2: Current project status. The system is positioned as research infrastructure and dataset tooling, not as a deployed drone autopilot or a trained VLA policy.

6 Validation

The current public package is released as `gs-dronegym==0.3.0` on PyPI and includes command-line tools for environment viewing, dataset generation, dataset validation, behavior cloning, and benchmark evaluation. A CPU-compatible mock-renderer path is kept as the default testable route. On the development machine used for this draft, the test suite reports `38 passed` with two warnings related to optional `gsplat`/CUDA extension checks.

The validation in this paper should be interpreted as software-artifact validation, not as evidence of VLA policy superiority. The implemented checks cover dynamics behavior, renderer interfaces, task reset/success/failure logic, environment API compatibility, metrics, schema round-tripping, benchmark adapters, live viewer behavior, behavior-cloning smoke tests, and dataset generation/-validation.

6.1 Artifact Status

Table 2 summarizes the current state of the released artifact. The table is intentionally phrased as implementation status rather than empirical validation.

6.2 End-to-End Dataset and Baseline Study

To verify that the dataset-generation and learning interfaces compose, we ran a small CPU mock-rendered baseline study. This experiment is intentionally not presented as a competitive learning result. It tests whether a user can generate data, validate it, train the included behavior-cloning baseline, compute held-out imitation metrics, and evaluate the resulting checkpoint through the benchmark interface.

6.3 Local Resource and Baseline Measurements

We additionally measured the CPU/mock path on a Windows workstation with an AMD Ryzen 7 7735HS CPU, 16 GB RAM, and Python 3.11. These measurements are useful for reproducibility because the mock path is the default installation path and does not require CUDA, `gsplat`, or scene downloads. They should not be read as 3DGS rendering benchmarks.

Step	Result	Interpretation
Generate synthetic dataset	24 episodes, 593 steps	Data factory writes Parquet and media artifacts.
Validate dataset	valid	Validator resolves schema, media paths, stages, tasks, and labels.
Train BC for three epochs	159 train examples, loss 0.0332	Baseline training loop and checkpoint export run.
Evaluate BC imitation	val MSE 0.1829	Held-out imitation error remains high.
Evaluate BC on PointNav	0/5 success, collision rate 1.0	Small mock dataset does not yield a useful live policy.

Table 3: Small end-to-end baseline study. The result supports pipeline functionality and exposes the difficulty of learning useful navigation from very small mock-rendered datasets.

Measurement	Result	Notes
Mock renderer throughput	134.5 FPS	224×224 RGB/depth frames
Mock renderer latency	7.44 ms/frame	deterministic CPU renderer
Full environment throughput	51.7 steps/s	includes dynamics and rendering
Full environment latency	19.4 ms/step	PointNav, random actions

Table 4: Local CPU/mock-path throughput. The script `paper/run_local_measurements.py` regenerates these numbers.

6.4 Missing Empirical Evidence

The present artifact validation leaves several research questions open. A publication-grade empirical study should add: (1) imitation-learning or reinforcement-learning baselines on larger generated datasets; (2) ablations over mock versus 3DGS rendering, curriculum stages, and safety labels; (3) real `gsplat` throughput and memory measurements on released Gaussian scenes; (4) comparisons to existing simulators where possible; and (5) held-out real-scene or real-drone transfer experiments. We keep these as explicit future work rather than implying that the current package already demonstrates transfer.

7 Limitations

GS-DroneGym has several important limitations. First, it is not a real-drone flight stack. It does not integrate with PX4, ROS 2, Betaflight, motion-capture systems, or onboard autonomy pipelines. Second, the collision model is simplified and task-centric rather than a high-fidelity physical contact model. Third, real 3D Gaussian scene creation depends on external reconstruction tools such as Nerfstudio and on platform-specific GPU support. Fourth, the VLA-AN-like dataset factory is an approximation based on public design ingredients; it is not an exact reproduction of

Policy	Episodes	Success	Collision
Zero action	5	0/5	0/5
Random action	5	0/5	5/5
Three-epoch BC	5	0/5	5/5

Table 5: Tiny mock PointNav baselines. These are negative controls for the evaluation interface, not competitive policy results. The zero policy stays alive but does not solve the task; random and undertrained BC policies commonly collide.

Priority	Required addition before archival submission	Current status
High	BC and RL baselines on large synthetic datasets with held-out tasks/scenes	not yet done
High	Consent-cleared public Gaussian indoor scene and reconstruction recipe	public archives linked; no bundled small asset
Medium	Real 3DGS FPS, memory, and quality measurements	pending local GPU run
Medium	Sensor-noise validation or explicit calibration protocol	augmentation only
Low	Additional diagrams, notebooks, and appendix cleanup	partially done

Table 6: Publication-readiness checklist. This table separates the current technical-report artifact from the stronger empirical package needed for a venue submission.

VLA-AN's internal dataset. Fifth, this draft does not report trained VLA policy results, real-world deployment, or statistically meaningful benchmark comparisons.

These limitations define the next evaluation steps. Stronger future evidence should include real Gaussian scenes from multiple environments, held-out scene splits, baseline policy training, ablations over rendering and safety labels, and real-drone transfer experiments if a flight stack is later integrated.

Real-scene asset status. The code can consume local Gaussian `.ply` scenes through `GSplatRenderer`. The built-in scene registry now points to public NerfBaselines Gaussian Splatting archives[1] for MipNeRF360 and Tanks and Temples scenes, and the loader can cache a downloaded archive and extract a contained `.ply`. However, this draft still does not bundle a small consent-cleared GS-DroneGym-owned scene. This remains an artifact gap for readers who want a lightweight photorealistic test without multi-gigabyte downloads. A complete release should include at least one small indoor Gaussian scene, license and privacy notes for the capture, the exact Nerfstudio export command, and a checksum or dataset card for the resulting `.ply`.

8 Reproducibility and Artifacts

The code is released at `https://github.com/09Catho/gs-dronegym` and the package is available from PyPI as `gs-dronegym`. The package requires Python 3.10 or newer. The core CPU path can

[1] `https://huggingface.co/nerfbaselines/nerfbaselines`

be installed with `pip install gs-dronegym`. Appendix A lists the exact commands used for the smoke study and for common user workflows.

Optional real-scene rendering requires a compatible GPU environment, `gsplat`, and a Gaussian `.ply` scene. Built-in public scene handles resolve to large external Gaussian archives, while local user-captured scenes can be passed as direct `.ply` paths. Releasing a small consent-cleared indoor scene owned by this project and documenting a robust Nerfstudio-to-`GSplatRenderer` workflow remains a high-priority next step.

The local measurements in Tables 4 and 5 were generated with:

```
python paper/run_local_measurements.py --frames 100 --steps 100 --episodes 3 --output
    paper/local_measurements.json
```

The dataset and BC study in Table 3 was generated with:

```
python paper/run_paper_experiments.py --dataset-root outputs/paper_baseline_dataset --
    scenes mock://paper_a mock://paper_b mock://paper_c --episodes-per-scene 8 --width 96
     --height 96 --renderer-device cpu --allow-mock-rendering --epochs 3 --batch-size 16
    --eval-episodes 5 --checkpoint outputs/paper_baseline_policy.pt --output paper/
    paper_experiment_results.json --force
```

9 Ethics and Safety

GS-DroneGym is intended for simulation, dataset generation, and research prototyping. It should not be treated as a safety-certified autonomy stack or used to fly physical drones without a separate flight-control, safety, and compliance layer. Datasets generated from scans of real rooms or laboratories may encode private geometry, visual content, or sensitive locations; such scenes should be collected with consent, stored carefully, and released only when privacy and licensing constraints are satisfied. Because the system studies autonomous navigation, misuse risks include unsafe drone operation and privacy-invasive mapping. These risks reinforce the need to keep physical deployment out of scope unless proper safety engineering is added.

10 Conclusion

GS-DroneGym provides open infrastructure for drone VLA research by connecting simulated quadrotor dynamics, waypoint-style control, photorealistic rendering interfaces, benchmark tasks, and synthetic dataset generation. Its current role is to make aerial VLA environment and data experiments easier to reproduce and inspect. The next research step is empirical: generate larger real-scene datasets, train baseline policies, evaluate held-out scenes and tasks, and measure whether 3DGS-based visual data improves transfer relative to mock or conventional synthetic rendering.

A Reproducibility Commands

Install the package:

```
pip install gs-dronegym
```

Run a minimal environment smoke test:

```
python -c "import gs_dronegym; env = gs_dronegym.make('PointNav-v0', scene=None); obs,
    info = env.reset(seed=0); print(obs['rgb'].shape, obs['depth'].shape)"
```

Launch the interactive viewer:

```
gs-dronegym-live-view --env-id PointNav-v0 --scene None --policy keyboard --action-mode
    waypoint
```

Generate and validate the smoke dataset used in Table 3:

```
gs-dronegym-generate-dataset outputs/paper_smoke_dataset --scenes mock://paper_a mock://
    paper_b --episodes-per-scene 6 --renderer-device cpu --allow-mock-rendering
gs-dronegym-validate-dataset outputs/paper_smoke_dataset
```

Train and evaluate the one-epoch behavior-cloning smoke baseline:

```
gs-dronegym-train-bc outputs/paper_smoke_dataset --format gs_dronegym --epochs 1 --split
    train --checkpoint outputs/paper_smoke_policy.pt
gs-dronegym-evaluate --benchmark gs_dronegym --env-id PointNav-v0 --scene None --policy
    outputs/paper_smoke_policy.pt --n-episodes 3
```

Run with a real Gaussian scene after exporting a compatible `.ply`:

```
gs-dronegym-live-view --env-id PointNav-v0 --scene C:\path\to\scene.ply --renderer-device
     cuda --policy keyboard
```

References

[1] Remi Cadene, Simon Alibert, Alexander Soare, Quentin Gallouedec, Adil Zouitine, Steven Palma, Pepijn Kooijmans, Michel Aractingi, Mustafa Shukor, Dana Aubakirova, Martino Russi, Francesco Capuano, Caroline Pascal, Jade Choghari, Jess Moss, and Thomas Wolf. Lerobot: State-of-the-art machine learning for real-world robotics in pytorch. `https://github.com/huggingface/lerobot`, 2024. Accessed 2026-04-15.

[2] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkuehler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), 2023.

[3] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. Openvla: An open-source vision-language-action model, 2024.

[4] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning, 2023.

[5] Valerii Serpiva, Artem Lykov, Artyom Myshlyaev, Muhammad Haris Khan, Ali Alridha Abdulkarim, Oleg Sautenkov, and Dzmitry Tsetserukou. Racevla: Vla-based racing drone navigation with human-like behaviour, 2025.

[6] Yuze Wu, Mo Zhu, Xingxing Li, Yuheng Du, Yuxin Fan, Wenjun Li, Zhichao Han, Xin Zhou, and Fei Gao. Vla-an: An efficient and onboard vision-language-action framework for aerial navigation in complex environments, 2025.